

REVIEW

Nezha-Arc Independent Code Review (§11.4.142 / §11.4.125)

Revision: 1 **Last modified:** 2026-06-16T18:14:10Z **Reviewer:** independent code-review agent (read-only, non-committing — NOT the author, NOT the committer)
Mode: READ-ONLY. No edit / stage / commit / git add performed (§11.4.84 working-tree quiescence respected — committer agent is sole committer). **Scope:** the 4 STABLE landed nezha-arc fixes ONLY: 5227585 (iter1), 2f326cd (iter2), 6a1aa8c (iter3), e198b17 (iter4). In-flight committer commits NOT reviewed.
Discipline: systematic-debugging mindset, §11.4.1 (no PASS/FAIL bluff), §11.4.6 (evidence-backed, no rubber-stamp), §1.1 (a real gate's mutation must FAIL the test).

Evidence captured this review

- go test ./pkg/api/ ./pkg/translator/llm/ ./internal/config/ ./cmd/api-server/ -count=1 → **all GREEN** (api 2.37s, llm 20.97s, config 0.94s, api-server 6.17s).
 - go build ./... → **EXIT 0** (clean).
 - Source read: internal/config/config.go (LoadConfig / loadAPIKeysFromEnv / ApplyEnvOverrides / DefaultConfig), pkg/translator/llm/llm.go (strip + IsKnownProvider + ValidModels), pkg/api/handler.go (createTranslator), pkg/api/verifier_handlers.go, cmd/api-server/main.go (probeBackendState).
 - §1.1 mutation proofs read: docs/qa/nezha_heavy_iter2_20260616T133401Z/red_baseline_and_mutation_proofs.txt (Finding#1 guard-removal FAILs, Finding#2 revert-to-one-shot FAILs, Finding#3 RED_MODE=1 FAILs).
 - Lockstep audit: every NewLLMTranslatorWithConfig factory case Provider* has a ValidModels entry → IsKnownProvider rejects nothing the factory can build (0 mismatches).
-

Fix 1 — 5227585 iter1: commentary strip + create-default env-override

1a. LLM commentary-contamination strip — GO

- **Correct + robust.** stripTranslationCommentary splits on `\n\n`, walks trailing blocks backward, stops at the first non-commentary block — it cannot eat mid-content, only trailing asides. Defense-in-depth (prompt instruction in both branches + CreatePromptForLanguages + `universal.go` + post-strip) is sound: even if a model ignores the prompt, the post-processor catches it.
- **Anti-bluff honesty (§11.4.6).** commentaryLeadMarkers + isCommentaryBlock are deliberately specific (won't eat a paragraph that merely starts with a common word). Markers are real captured live-nezha responses, not invented.
- **Test genuinely exercises + catches negation.** TestEnhanceTranslation_StripsModelCommentary uses LITERAL captured contaminated outputs and mustNotHas fragment asserts; reverting enhanceTranslation to enhanced := translated (the pre-fix line) makes every contaminated case FAIL. ..._DoesNotEatRealParagraphs is the false-positive guard (real multi-paragraph passes untouched) — proven by the GREEN run. Not a bluff gate.

1b. create-default config env-override skip — GO

- **Correct root-cause fix.** Confirmed loadAPIKeysFromEnv sets `c.Security.JWTSecret` from `JWT_SECRET`; DefaultConfig() ships `EnableAuth=true, JWTSecret=""`; the create-default branch previously skipped env-loading → `Validate()` "JWT secret is required" crash-loop. `cfg.ApplyEnvOverrides()` before `SaveConfig` closes it.
 - **Test catches negation.** TestApplyEnvOverrides_JWTSecretReachesDefaultConfig asserts the precondition (`EnableAuth=true`, empty `JWTSecret`) then that `ApplyEnvOverrides` makes `Validate` pass — removing the `ApplyEnvOverrides()` call (or the body) makes it FAIL.
 - **MINOR (non-blocking, naming honesty).** `ApplyEnvOverrides()` only calls `loadAPIKeysFromEnv()`, whereas `LoadConfig` does `applyDefaults()` + `loadAPIKeysFromEnv()`. For the create-default path this is harmless (`DefaultConfig` already IS the defaults, so `applyDefaults` is a no-op there). But the name implies broader scope than it has; a future caller could wrongly assume parity with `LoadConfig`. Suggest a committer follow-up: either rename to `applyEnvOverrides`/document the scope, or have it also call `applyDefaults()` for true `LoadConfig`-parity. Not release-blocking.
-

Fix 2 — 2f326cd iter2: verifier honest-503 + active gRPC health probe + env-file deploy

2a. Verifier routes always-register + honest 503 — GO

- **Correct + §11.4.69-aligned.** Routes always registered; requireEnabled gin middleware returns honest 503 reason=llmsverifier_disabled when off, real data when on. InitVerifierFromConfig wires SetEnabled(cfg.LLMsVerifier.Enabled). Replacing a misleading 404 ("API doesn't exist") with a truthful 503 is the right contract.
- **Mutation-proven (§1.1).** Evidence file: removing the requireEnabled guard makes TestVerifier_DisabledRoutes_Return503NotFound FAIL (served instead of 503). The enabled-path guard (TestVerifier_EnabledRoutes_PassThroughGuard) proves SetEnabled(true) does not short-circuit. Real gate.

2b. api-server /health active gRPC probe — GO

- **Correct.** gRPC NewClient conns are lazy/IDLE until first RPC; the one-shot GetState() reported 503 on a reachable backend. probeBackendState does Connect() + bounded WaitForStateChange (2s) — IDLE-reachable→READY (healthy), IDLE-down→TRANSIENT_FAILURE (unhealthy). Never blind-trusts IDLE (avoids false-positive on a genuinely down backend). Timeout uses the request context, bounded — won't hang / health.
- **Test catches negation.** TestHealthCheck_LazyIdleButReachableBackend_ReportsHealthy boots a REAL in-process gRPC server (genuinely reachable, lazy-IDLE) and asserts 200/READY; mutation proof: reverting to one-shot GetState() FAILs it (503 IDLE). The t.Skipf on already-Ready precondition is honest (§11.4.3) — it does not falsely PASS. Real gate.

2c. compose --env-file deploy — GO (deploy script, lower risk)

- scripts/nezha-deploy.sh always passes --env-file .env.nezha; guard test nezha_deploy_envfile_guard_test.sh RED_MODE=1 flags a no-env-file invocation (mutation-proven). Honest §11.4.108 note in the commit re: podman-compose 1.5.0 stale-image gap (build+stop+rm+up) — accurately disclosed, not bluffed.

Fix 3 — 6a1aa8c iter3: reject unsupported provider 400 + §11.4.120 reconciliation — GO

- **Correct + §11.4.69 no-silent-substitution.** createTranslator rejects ! llm.IsKnownProvider(providerName) with 400 "unsupported provider" BEFORE the bridge, so a caller asking for a non-existent provider no longer silently gets a real translation from a DIFFERENT provider (response-correctness defect: ask X get Y).

- **Lockstep VERIFIED (independent audit, not taken on trust).**
IsKnownProvider derives from ValidModels; I diffed the factory switch cases against ValidModels keys — **every factory-buildable provider has a ValidModels entry (0 in-factory-but-not-registry mismatches)**. So the guard rejects nothing the system can actually build. "" correctly excluded (means "default", handled above).
 - **Default-fallback path safe.**
DefaultConfig().Translation.DefaultProvider = "openai" (a known provider) → provider-omitted requests do not trip the 400 out-of-the-box. Confirmed.
 - **§11.4.120 reconciliation is genuine, NOT fake-pass/revert.** Stale pre-bridge 400 "Will fail at translator creation" expectations in handler_test/batch_translate_test were rewritten to the corrected behaviour: valid known provider → 200 (deterministic via installMockBridge, §11.4.98/§11.4.50), unsupported → 400 on BOTH translateText and batch paths. The new unsupported-provider cases assert 400 + "unsupported provider" — removing the guard in handler.go makes them FAIL. Real gate.
 - **BEHAVIORAL NOTE (non-blocking).** If an operator misconfigures default_provider to empty/unknown, ALL provider-omitted requests now 400. This is fail-loud-on-misconfig (preferable to silent substitution) and batch_handlers_test.go:735 already covers a "nonexistent" default. Acceptable; worth a one-line operator-doc note.
 - **PENDING honestly disclosed (§11.4.6/§11.4.108).** Commit states the nezha live-server rebuild+reboot+sink-side re-validation is NOT yet done (live image runs pre-fix binary). This is an honest pending, correctly flagged — **the committer MUST close it before the arc is release-tagged** (see release-readiness below). e198b17's commit body shows 443-port sink-side GREEN/RED fixtures for the unsupported-provider contract, partially covering this.
-

Fix 4 — e198b17 iter4: parenthetical style/dialect aside escape — GO with a watch-item

- **Correct.** iter1's isCommentaryBlock only treated a trailing ()/[] block as commentary when its inner text started with "note" OR contained "translat". A live ru→sr (Using Ekavica dialect ... as per guidelines) had neither → English meta leaked into Serbian. Fix widens with commentaryParenSignalWords (dialect/vocabulary/guideline/register/using/as per/i used/i chose/...). This is a legitimate §11.4.4 test-interrupt → §11.4.146 extend-to-all-cases follow-up, not a re-bug.
- **Test catches negation + false-positive guarded.** Two live-captured RED fixtures (sr_paren_dialect_aside, en_bracket_style_aside) with mustNotHas; reverting the signal-word loop makes them FAIL. ..._KeepsBenignTrailingParenthetical ("(I bio je veoma umoran.)" — real content, no signal word) proves no over-strip. Real gate.
- **WATCH-ITEM (§11.4.6 residual false-positive surface, non-blocking).** The signal words register, vocabulary, using, i kept are ordinary words that COULD appear inside a genuine trailing parenthetical of real translated prose (e.g. a quote discussing language, or a sentence ending in a parenthetical containing "register"). The block must be FULLY enclosed in ()/[] AND trailing AND \n\n-separated, which makes a real-content false strip unlikely

but not impossible. Recommend the committer add one more benign-content guard fixture containing a signal word inside genuine prose (e.g. a trailing parenthetical that legitimately uses "register" as a noun) to pin the boundary, OR document the accepted residual risk. Current corpus does not yet prove this edge. Not release-blocking given the FULLY-enclosed+trailing constraint, but it is the one place the stripper could in principle eat real content.

Cross-cutting checks

- **No solve-A-create-B (§11.4.1).** Each fix is additive/guarded; no fix re-introduces another's defect. iter4 strictly widens iter1 (same function, more coverage) — verified iter1's fixtures still pass in the GREEN run.
- **No bluff gates (§11.4.1/§1.1).** Every reviewed test is mutation-proven (iter2 via the captured proof file; iter1/iter3/iter4 by reading the assertions + confirming the pre-fix line/guard removal forces FAIL). None pass on broken code.
- **§11.4.69 honesty.** Honest 503 (not 404), honest 400 (not silent substitution), honest IDLE-probe (not blind-trust) — all three are no-silent-substitution wins.
- **Tests are fully automated/re-runnable (§11.4.98).** All use httptest / in-process gRPC / env-only; no manual intervention. GREEN at -count=1.

Per-fix verdict

Fix	Verdict
5227585 iter1 (commentary strip + env-override)	GO (1 MINOR naming follow-up)
2f326cd iter2 (503 + health probe + env-file)	GO
6a1aa8c iter3 (400 unsupported provider)	GO (live re-validation PENDING — must close)
e198b17 iter4 (paren aside escape)	GO (1 WATCH-ITEM false-positive fixture)

Overall release-readiness — GO for the arc, with TWO non-blocking committer follow-ups + ONE blocking-before-TAG item

The four fixes are correct, robust, anti-bluff, and their tests genuinely catch the negation (no bluff gates). `go build ./... clean`; the four affected test packages GREEN. The arc materially improves end-user correctness (clean translations, honest API error codes, accurate health).

Blocking BEFORE a release TAG (not blocking the commits themselves):

1. **iter3 live nezha sink-side re-validation (§11.4.108).** iter3 honestly states the live server still runs the pre-fix binary. Per §11.4.108 / §11.4.130, the committer must rebuild→reboot→sink-side re-validate the unsupported-provider 400

contract on live nezha before the arc is tagged. (e198b17's 443-port RED/GREEN fixtures already partially evidence this — confirm they reflect the rebuilt image.)

Non-blocking committer follow-ups: 2. **iter1** — clarify `ApplyEnvOverrides()` scope vs `LoadConfig` (name implies more than it does); either rename/document or add `applyDefaults()` for true parity. 3. **iter4** — add one benign-content fixture with a signal word inside genuine trailing-parenthetical prose to pin the false-positive boundary, or document the accepted residual.

No defects, no bluff-capable gates, no silent substitutions found. This is an evidence-backed GO, not a rubber-stamp.